

Practical 1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

✓ step 1: Load (iris.csv) Dataset

```
df = pd.read_csv("https://raw.githubusercontent.com/mwaskom/seaborn-data/master
```

```
# Display first fews rows
print("Dataset Head:\n", df.head())
```

Dataset Head:

	sepal_length	sepal_width	petal_length	petal_width	species
0	5.1	3.5	1.4	0.2	setosa
1	4.9	3.0	1.4	0.2	setosa
2	4.7	3.2	1.3	0.2	setosa
3	4.6	3.1	1.5	0.2	setosa
4	5.0	3.6	1.4	0.2	setosa

✓ step 2: Display features and their types

```
#count the number of features
num_features = df.shape[1] -1 #Excluding the target column
print(f"\nNumber of features: {num_features}")
```

Number of features: 4

```
#Feature information
print("\nFeature Types:")
print(df.dtypes)
```

Feature Types:

sepal_length	float64
sepal_width	float64
petal_length	float64
petal_width	float64
species	object
dtype:	object

Step 3: Compute and display Summary statistics for each feature available in the dataset.

(eg: min value, max value, mean, range, standard deviation, variance and percentiles)

```
#Summary Statistics
summary_stats = df.describe()
print("\nSummary Statistics:\n", summary_stats)
```

```
Summary Statistics:
      sepal_length  sepal_width  petal_length  petal_width
count      150.000000      150.000000      150.000000      150.000000
mean         5.843333         3.057333         3.758000         1.199333
std          0.828066         0.435866         1.765298         0.762238
min          4.300000         2.000000         1.000000         0.100000
25%          5.100000         2.800000         1.600000         0.300000
50%          5.800000         3.000000         4.350000         1.300000
75%          6.400000         3.300000         5.100000         1.800000
max          7.900000         4.400000         6.900000         2.500000
```

```
# Additional statistics: Range, Variance, Standard Deviation, and Percentiles
```

```
feature_columns = df.columns[:-1] # Exclude species column

range_values = df[feature_columns].max() - df[feature_columns].min()
variance_values = df[feature_columns].var()
std_dev_values = df[feature_columns].std()
percentiles = df[feature_columns].quantile([0.25, 0.50, 0.75])
```

```
print("\nRange of each feature:\n", range_values)
print("\nVariance of each feature:\n", variance_values)
print("\nStandard Deviation of each feature:\n", std_dev_values)
print("\nPercentiles (25th, 50th, 75th):\n", percentiles)
```

```
Range of each feature:
  sepal_length    3.6
  sepal_width     2.4
  petal_length    5.9
  petal_width     2.4
dtype: float64

Variance of each feature:
  sepal_length    0.685694
  sepal_width     0.189979
  petal_length    3.116278
  petal_width     0.581006
dtype: float64
```

Standard Deviation of each feature:

```
sepal_length    0.828066
sepal_width     0.435866
petal_length    1.765298
petal_width     0.762238
dtype: float64
```

Percentiles (25th, 50th, 75th):

	sepal_length	sepal_width	petal_length	petal_width
0.25	5.1	2.8	1.60	0.3
0.50	5.8	3.0	4.35	1.3
0.75	6.4	3.3	5.10	1.8

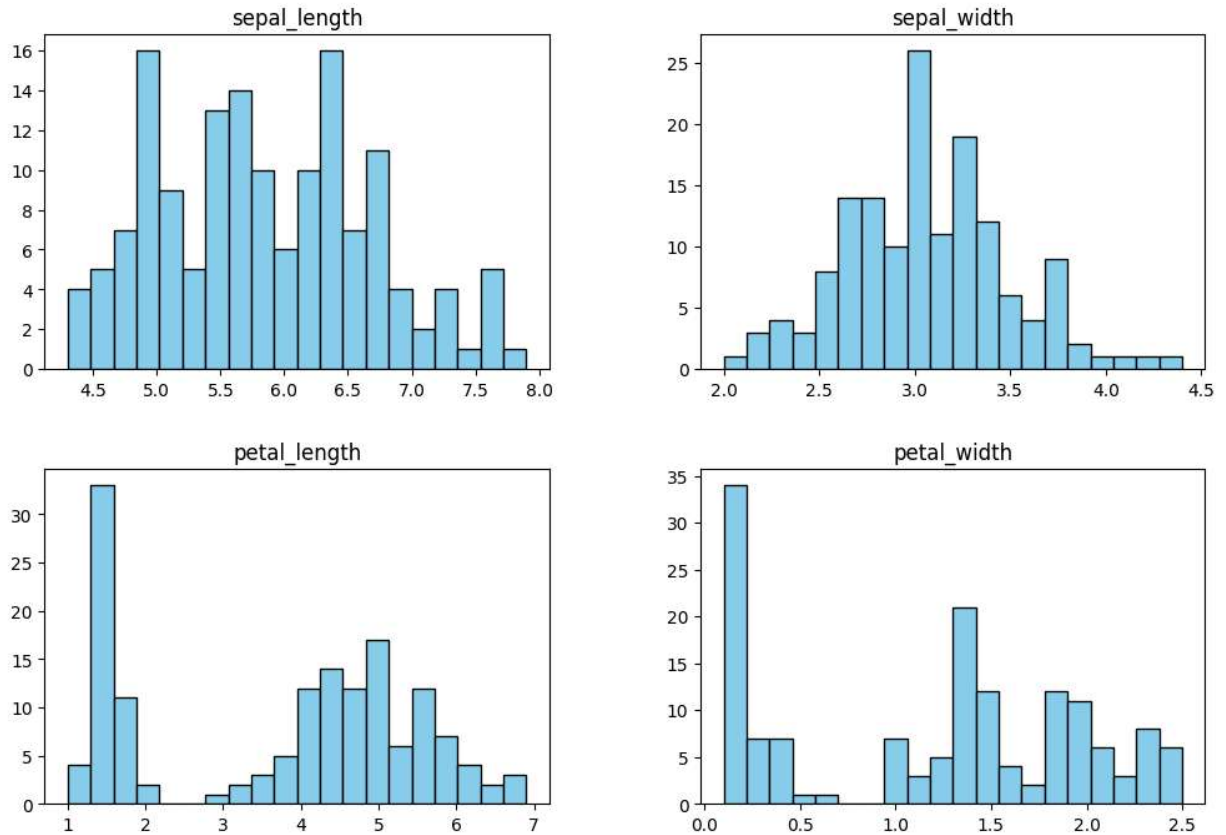
Step 4: Data Visualization - Create a histogram for each feature in the dataset to illustrate the feature distributions. Plot each histogram.

```
# Histograms for each feature
```

```
plt.figure(figsize=(10, 6))
df.hist(bins=20, figsize=(12, 8), grid=False, color='skyblue', edgecolor='black')
plt.suptitle("Histograms of Iris Dataset Features")
plt.show()
```

<Figure size 1000x600 with 0 Axes>

Histograms of Iris Dataset Features



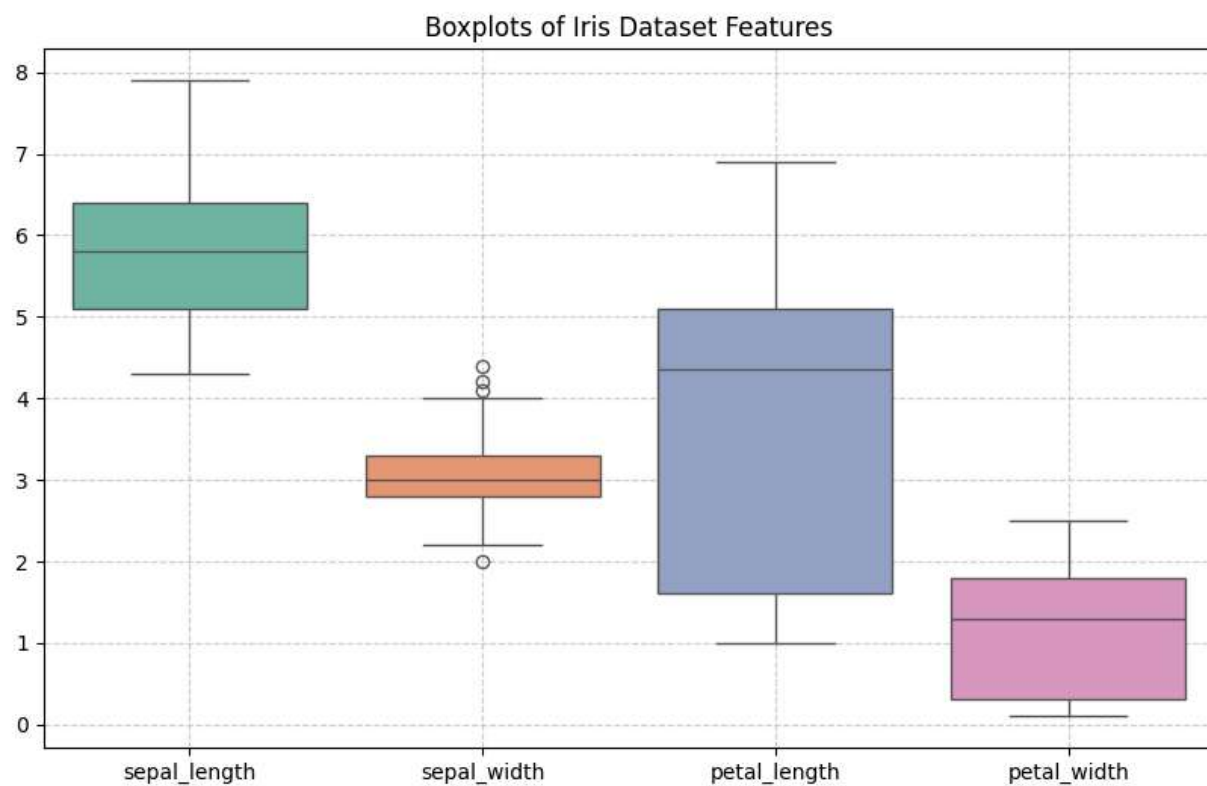
****Step 5: Create a boxplot for each feature in the dataset. All of the boxplots should be combined into a single plot. Compare distributions and identify outliers. ****

```
# Boxplots for each feature
```

```
plt.figure(figsize=(10, 6))
sns.boxplot(data=df.iloc[:, :-1], palette="Set2") # Excluding species column
plt.title("Boxplots of Iris Dataset Features")
```

```
plt.grid(True, linestyle='--', alpha=0.6)
```

```
plt.show()
```



#Identify Outliers using IQR

```
Q1 = df[feature_columns].quantile(0.25)
```

```
Q3 = df[feature_columns].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
outliers = ((df[feature_columns] < (Q1 - 1.5 * IQR)) | (df[feature_columns] > (Q3 + 1.5 * IQR)))
```

```
print("\nOutliers detected:\n", outliers.sum())
```